

Algorithmes et Pensée Computationnelle

Algorithmes spatiaux

Le but de cette séance est de comprendre les caractéristiques des données spatiales et les structures de données couramment utilisées pour les manipuler. Lors de cette séance, nous implémenterons des algorithmes de manipulation de structures de données spatiales présentés en cours. Au terme de la séance, l'étudiant sera en mesure d'utiliser plusieurs algorithmes spatiaux pour résoudre des problèmes de base de façon efficace.

1 Nearest-Neighbor

Dans cette section, nous allons implémenter une recherche du plus proche voisin. Le but de cette méthode est de trouver le voisin le plus proche du point de départ en tenant compte de ce point de "départ" et un ensemble de points. Nous allons implémenter cet algorithme et ensuite l'étendre à un algorithme des "k plus proches voisins" (recherche des k voisins les plus proches plutôt que du seul voisin le plus proche). Les points seront représentés par des tuples construits de la manière suivante : $\text{point} = (1, 2)$ où 1 est l'abscisse et 2 l'ordonnée.

Nous vous recommandons de traiter les questions dans l'ordre.

Question 1: (🕒 5 minutes) Python — La fonction de distance

Pour implémenter notre recherche, nous avons besoin d'écrire une fonction permettant de calculer la distance entre 2 points. Ecrivez une fonction qui permet de calculer la distance entre 2 points.

Question 2: (🕒 10 minutes) Python — Nearest-neighbor search

Implémentez la recherche du voisin le plus proche. Cette dernière fonctionne de la façon suivante :

1. Créez une fonction qui prend comme arguments une liste de points et un point de départ.
2. Gérez le cas où la liste de points est vide.
3. Traversez chaque point.
4. Pour chaque point, calculez la distance entre ce point et le point de départ.
5. Retournez un tuple contenant les coordonnées du point le plus proche sous forme d'un tuple et la distance — par exemple : $((x, y), \text{dist})$.

Question 3: (🕒 15 minutes) K-nearest-neighbor search

Améliorez l'algorithme du voisin le plus proche effectué plus haut afin qu'il puisse retourner des K -plus proches voisins.

2 K-dimensional tree

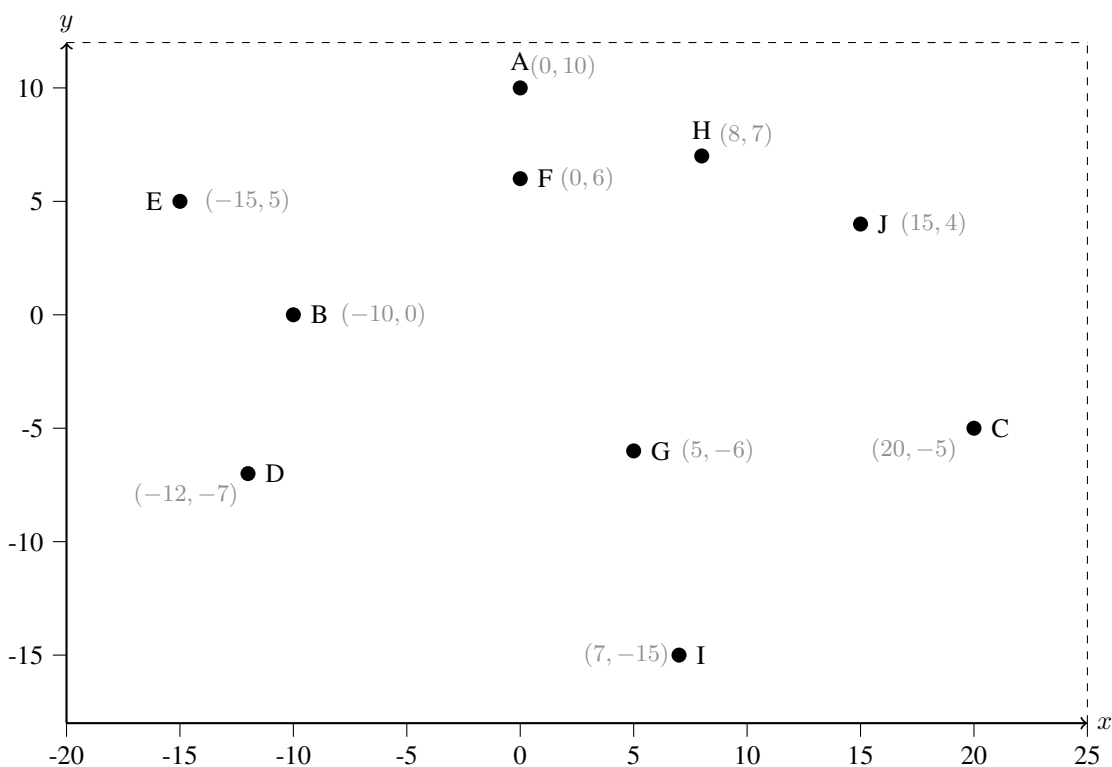
Question 4: (🕒 5 minutes) Lecture du code — Construction d'un KD-Tree

Question 5: (🕒 10 minutes) KD-Tree, un échauffement : Papier

Vous trouverez ci-dessous une liste de points numérotés de 1 à 10. Placez-les dans un KD-Tree et dessinez la séparation de l'espace qui en résulte.

Algorithm 1 BuildKDTree(points, depth, k)

```
1: if points =  $\emptyset$  then
2:   return NIL
3: end if
4: axis  $\leftarrow$  depth %  $k$                                 ▷ Select axis based on current depth
5: points  $\leftarrow$  SORT(points, key = axis)                  ▷ Sort by current axis
6: median_idx  $\leftarrow$  INTEGER(length(points)/2)           ▷ Index of median list item
7: median  $\leftarrow$  KNode(points[median_idx])
8: left  $\leftarrow$  points[1 ... median_idx - 1]
9: right  $\leftarrow$  points[median_idx + 1 ... length(points)]
10: median.left  $\leftarrow$  BuildKDTree(left, depth + 1,  $k$ )
11: median.right  $\leftarrow$  BuildKDTree(right, depth + 1,  $k$ )
12: return median
```

**⚠ Attention**

On ne construit pas l'arbre en appelant AddKDTree de manière itérative. On suit l'algorithme présenté dans la Question 4 qui est BuildKDTree.

Question 6: (🕒 15 minutes) **KD-Tree : Python**

L'objectif de cet exercice est d'écrire une fonction permettant d'ajouter un nœud à un KD-Tree. Les nœuds sont sous la forme $[(x, y), \text{enfant à gauche}, \text{enfant à droite}]$, x et y étant les coordonnées du nœud considéré. Chaque enfant peut être soit un nœud ou une feuille. Complétez le code contenu dans le fichier Question6.py.

Voici le pseudo-code permettant d'ajouter un nœud à un KD-Tree :

Algorithm 2 AddKDTree(node, point, k , cutaxis)

```
1: if node = NIL then
2:   new_node  $\leftarrow$  KDNode(point)
3:   return new_node
4: end if
5: if point[cutaxis]  $\leq$  node.point[cutaxis] then                                 $\triangleright$  Insert into the left (lesser) subtree
6:   new_cutaxis  $\leftarrow$  (cutaxis + 1) (mod  $k$ )
7:   node.left  $\leftarrow$  AddKDTree(node.left, point,  $k$ , new_cutaxis)
8: else
9:   new_cutaxis  $\leftarrow$  (cutaxis + 1) (mod  $k$ )
10:  node.right  $\leftarrow$  AddKDTree(node.right, point,  $k$ , new_cutaxis)
11: end if
12: return node
```

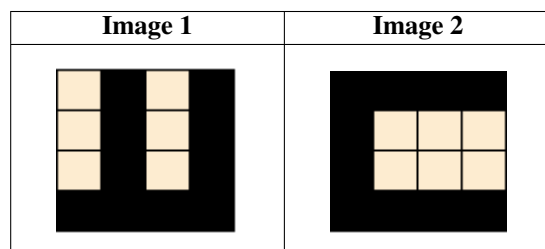
⚠ Attention

Afin de simplifier la question, vous ne devez pas créer une classe KDNode pour représenter les nœuds. Chaque nœud est représenté comme une liste. Donc, quand vous transformez le pseudocode, `node.point` sera `node[0]`, `node.left` sera `node[1]`, et `node.right` sera `node[2]`.

3 Quad-Tree

Question 7: (🕒 10 minutes) Une mise en train : Papier

Encodez les images ci-dessous dans un Quad-Tree.

**Question 8:** (🕒 10 minutes) Une mission capitale : Papier Optionnel

Récemment embauché par la CIA, vous êtes à la recherche d'un individu se cachant dans une des villes suivantes : Bleu, Orange, Noir, Gris, Vert et Jaune. Votre mission, si vous l'acceptez, est de créer un Quad-Tree qui vous permettra de géolocaliser le criminel de façon efficace. Vous trouverez ci-dessous une carte de villes. Créez le Quad-Tree et rétablissez la justice.

